Pretraining

Lecture 2: data, training and scale

Aarne Talman

29 October 2026

LDA-T318 | LLMs and Generative AI for NLP | University of Helsinki

Course overview

Wk	Dates	Tuesday	Thursday	Teacher
1	**27 / 29 Oct**	**Language models**	**Pretraining**	**Aarne**
2	3 / 5 Nov	Post-training	Lab 2: tune vs prompt	Aarne
3	10 / 12 Nov	Inference	Lab 3: serve	Aarne
4	17 / 19 Nov	Evaluation	Lab 4: evaluate	Jussi
5	24 / 26 Nov	RAG	Lab 5: RAG	Dmitry
6	1 / 3 Dec	Agents	Lab 6: agent	Dmitry
7	8 / 10 Dec	Course recap	Special topics (TBC)	All

Reusing a task and model across labs is recommended, not required.

There are no intermediate submissions; the final report and code archive determine the grade.

Lecture overview

1. The training loop *8 min*
2. Data *25 min*
3. Scaling laws *10 min*
4. Training at scale *7 min*
5. Mid-training *10 min*
6. What pretraining buys *10 min*
7. Lab 1 briefing *8 min*

Take away

By the end of today you should be able to explain how a language model is pretrained, make a rough compute estimate, and identify the main choices in its data recipe.

The training loop

What does pretraining ask the model to do?

$$\mathcal{L}(\theta) = -\frac{1}{T}\sum_{t=1}^{T} \log p_\theta(x_t \mid x_{<t})$$

In words: at every position, the model predicts the next token. It receives a larger penalty when it assigns a low probability to the token that actually follows.

During training, the model sees the **true preceding tokens**. Every position is therefore a training example, and all positions in a sequence can be processed in parallel.

A sequence of 4096 tokens provides thousands of prediction examples in one pass through the model.

Take away

No manual labels are needed: the text supplies both the input and the correct next token.

One training step

1. **Predict.** Run a batch of text through the model and predict the next token at every position.
2. **Measure the error.** Compare the predictions with the actual next tokens and calculate one average loss.
3. **Work out what to change.** Backpropagation measures how each model weight contributed to that loss.
4. **Update the model.** The optimiser makes a small adjustment to the weights.

Repeat these steps over many batches. If training is working, the model gradually assigns more probability to the tokens that occur in its data.

Why transformers made large-scale training practical

Predictions for all positions in a training sequence can be computed in parallel. An RNN must process the sequence one position at a time.

A BATCH CONTAINS MANY TOKEN PREDICTIONS

A **batch** is the text processed before one model update. In language-model training, its size is most clearly reported as a number of tokens.

$$32 \text{ sequences} \times 2048 \text{ tokens} = 65{,}536 \text{ tokens per step}$$

Why use batches?

- many predictions can run in parallel on a GPU
- averaging over examples gives a more reliable update

What limits their size?

- longer sequences and larger batches use more memory
- the practical choice depends on the available hardware

Why this matters: token batch size connects the training code to GPU use and to the total number of tokens seen during a run.

How do we know whether training is working?

Training loss

- should fall quickly at first, then more slowly
- shows how well the model predicts its training data

Validation loss

- is measured on text excluded from training
- should usually follow the same downward trend
- a growing gap can indicate memorisation

What the curves are used for

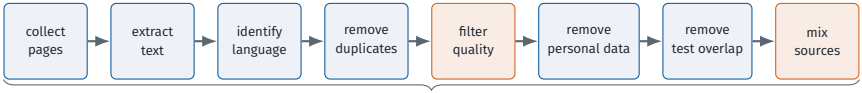
They help us check that the run is learning, compare checkpoints from the same run, and notice data or configuration problems early.

Compare like with like

Loss values are directly comparable only when the models use the same tokenizer and comparable data. A lower loss across different setups does not automatically mean a better language model.

Data

The pipeline



The raw crawl is not yet suitable for training. Each stage removes or transforms some material, but every decision can also remove useful text. The right choices depend on the languages and tasks the model should support.

Take away

Two models with the same architecture can learn different things because their training data was prepared differently.

Stage 1: collect web pages

A **web crawl** is a large collection of downloaded web pages. **Common Crawl** is a non-profit project that has collected snapshots of the web roughly monthly since 2008. Each snapshot contains billions of pages.

It provides two useful versions of each page:

- **WARC**: the original response, including HTML and headers
- **WET**: plain text extracted by Common Crawl

WET is easier to use. WARC gives you control over text extraction and usually produces cleaner training data.

What can go wrong?

If you use WET, you also accept Common Crawl's decisions about which parts of the page count as text. Navigation bars, cookie notices and link lists may remain. Recent datasets such as FineWeb, RefinedWeb, Dolma and HPLT therefore start from **WARC** and extract the text themselves. The crawl is also not a representative sample of all writing. Public, easily downloaded pages are over-represented; paywalled and login-protected text is under-represented.

Source: CommonCrawl; Penedo et al., *The FineWeb Datasets*, 2024; Penedo et al., *RefinedWeb*, 2023.

Stage 2: text extraction

Goal: turn an HTML page into clean text in the correct reading order.

Remove: navigation, sidebars, cookie notices, “related articles”, comment spam, licence footers, JavaScript.

Keep: the body text, in reading order, with paragraph and list structure intact.

trafilatura is one widely used extraction tool. FineWeb compared Common Crawl’s WET text with trafilatura applied to WARC pages. Models trained on the latter scored better, but extracting the text required more time and computing resources.

What can go wrong?

Extraction errors are often **systematic**: the same mistake is repeated on every page using a particular website template. For example, an extractor might remove the first paragraph from every page on a large content-management system. A small random sample may not reveal the problem because the remaining text still looks plausible.

Source: Penedo et al., *The FineWeb Datasets*, 2024; Barbaresi, *trafilatura*, 2021.

Stage 3: language identification

A language classifier, often fastText, predicts a language and a confidence score for each document. The pipeline keeps a document only if the score is above a chosen threshold.

Problem	What happens
short documents	unreliable under about 20 words, so short texts vanish
code-switching	a Finnish post quoting English clears neither threshold
related languages	Danish/Bokmål and Czech/Slovak get confused
threshold choice	a higher threshold gives cleaner but less data

FineWeb used a confidence threshold of 0.65 for English. A smaller language may need a lower threshold because much less text is available.

Source: Joulin et al., fastText, 2016; Penedo et al., *The FineWeb Datasets*, 2024.

Stage 4: remove duplicate text

Pipelines usually remove duplicates at three levels:

- **exact duplicates**: identical documents, such as mirrors and reposts
- **near-duplicates**: documents that are mostly the same, such as templated or lightly edited pages
- **repeated lines**: boilerplate that appears across many pages, such as menus and legal notices

Repeated text receives more weight during training simply because it occurs more often. Removing duplicates reduces unwanted memorisation and prevents boilerplate from dominating the corpus.

More deduplication is not always better

FineWeb obtained **worse** models when it removed duplicates across all 96 Common Crawl snapshots than when it processed each snapshot separately. Useful text is often copied across sites or remains online for years. Removing every repeated copy can therefore remove a useful signal of quality while retaining unique, low-quality pages. The deduplication strategy must be tested, not assumed.

Source: Lee et al., *Deduplicating Training Data Makes LMs Better*, 2022; Penedo et al., *The FineWeb Datasets*, 2024.

Stage 5a: rule-based quality filtering

Many web corpora use simple rules to decide whether to **keep or discard each document**. Two influential examples are **C4**, the web corpus used to train T5, and the data pipeline described for DeepMind's **Gopher** language model.

Example criteria for keeping a document

- its length is between 50 and 100 000 words
- the mean word length is 3–10 characters
- at least 80% of words contain a letter
- most lines end with sentence punctuation
- fewer than 30% of lines end in an ellipsis
- it has few symbols and no blocked terms

What are the rules trying to remove?

Pages dominated by menus, code, random symbols, spam or other text that does not resemble ordinary prose. But the rules are often **English-shaped**.

Word-length criteria can penalize Finnish or German compounds, while punctuation criteria may not fit Thai. Blocklists can also remove legitimate text, including sex education and writing by or about minority groups.

Exact rules differ between pipelines. They are cheap and auditable, but they can discard useful text and keep polished-looking spam.

Source: Raffel et al., *C4*, 2020; Rae et al., *Gopher*, 2021; Dodge et al., *Documenting the English C4*, 2021.

Stage 5b: quality filtering by classifier

A classifier can learn which documents to keep. **FineWeb-Edu** provides one example:

1. A large language model rates 460,000 documents from 0 to 5 for *educational value*.
2. Train a much smaller classifier to imitate those ratings.
3. Apply it to all 15 trillion tokens in FineWeb.
4. Keep documents with a score of 3 or higher.

Result: **1.3 trillion tokens** remain. With the same training budget, this subset produces better results on knowledge and reasoning benchmarks.

The trade-off

The model that produced the ratings now defines "good data". "Educational value" favours textbooks, tutorials and encyclopaedic prose over dialogue, fiction and informal writing. This may improve knowledge benchmarks but make the corpus less useful for conversational or creative tasks.

Source: Lozhkov et al., *FineWeb-Edu*, 2024; Penedo et al., *The FineWeb Datasets*, 2024.

How do we know which pipeline is better?

Looking at samples is not enough

Samples and corpus statistics reveal obvious problems. The stronger test is to train comparable small models on data from each pipeline and evaluate them on the same tasks.

A controlled comparison

- fix the architecture, tokenizer, hyperparameters and token budget
- train a 1.8B-parameter model on 28B tokens from each candidate dataset
- evaluate on a fixed suite of benchmarks chosen in advance for low variance and a signal above chance at that scale
- change **one** pipeline decision at a time

Source: Penedo et al., *The FineWeb Datasets*, 2024.

Why train a model to compare data?

Two datasets can both look reasonable to a person yet teach a model different things. This method isolates the effect of one data-processing decision. It is also expensive: every filter you want to test requires another training run.

Stage 6: remove personal information (PII)

Personally identifiable information (PII) includes email addresses, phone numbers, IP addresses, postal addresses and identity numbers.

Pattern matching finds structured details; named-entity recognition finds names and addresses. Detected text can be replaced with a placeholder so the rest of the sentence remains usable.

This matters for two reasons:

- **legal**: GDPR requires a lawful basis, data minimisation and appropriate safeguards
- **model behaviour**: a model may memorise and reproduce contact details from its training data

Source: Soldaini et al., *Dolma*, 2024; FineWeb-PII, 2024.

What can go wrong?

The tools can miss personal information and remove text that is not personal information. They may miss obfuscated details, mistake version numbers for IP addresses, or mistake a Finnish place name for a person's name. Removing obvious PII reduces risk, but it does not guarantee privacy. Context can still reveal a person's identity.

Stage 7: remove overlap with test data

Decontamination removes benchmark or test examples from the training corpus.

How: compare training documents with each test set. An *n*-gram is a sequence of *n* tokens. A long match suggests copying, so the matching passage or document is removed.

Why it is hard

- benchmarks must be known before training
- paraphrases and translations may not share the same words
- future benchmarks or later training can introduce new overlap

Source: Brown et al. 2020; Soldaini et al., *Dolma*, 2024.

Why it matters

If a model saw test questions during training, its score may measure memory rather than generalisation. In your project, keep the test split separate from prompt design, training and model selection. Use it only for the final evaluation.

Stage 8: choose the data mixture

The final step decides how much data to draw from each source. This strongly affects the skills and knowledge the model develops.

Source	Illustrative share	Why include it?
filtered web	60–85%	broad coverage and varied writing styles
code	5–20%	programming and structured problems
maths	2–10%	arithmetic and multi-step reasoning
books, papers	2–10%	long documents and formal writing
curated reference	1–5%	reliable facts and definitions

These shares vary considerably between models. **Upsampling** means drawing from a small source more often, so the model sees some documents more than once. Moderate repetition can help; too much can cause memorisation and reduce the value of further training.

Source: Muennighoff et al., *Scaling Data-Constrained Language Models*, 2023; Aryabumi et al., *To Code or Not To Code?*, 2024.

A SMALLER LANGUAGE HAS A SMALLER DATA BUDGET

- English is roughly **45%** of CommonCrawl documents; Finnish roughly **0.2%**
- after filtering, usable Finnish web text is on the order of **tens of billions** of tokens

A compute-optimal 7B model uses about 140B tokens, and many recent recipes use substantially more. Finnish data alone is therefore not enough for this kind of general-purpose model.

The consequence

A practical approach is a **multilingual** model in which Finnish is upsampled. This supplies enough varied training text while giving Finnish a larger share than its raw web frequency. It can also transfer useful patterns learned from English and code to Finnish contexts.

Source: CommonCrawl language statistics; TurkuNLP Finnish web corpora; Luukkonen et al., *Poro 34B*, 2024.

Ways to increase training data for a smaller language

Approach	What it provides	Main limitation
multilingual mixture	more total data and cross-lingual transfer	less compute spent directly on the target language
repeat scarce data	greater weight for existing target-language text	diminishing returns and more memorisation
machine translation	large volumes from high-resource languages	translation patterns and MT errors
synthetic text	controllable topics and formats	generator bias, errors and licence constraints
parallel text	explicit links between languages	limited volume and domain coverage

These approaches are usually combined. Poro, for example, mixed Finnish with English and code, repeated Finnish data, and included aligned English–Finnish text.

Source: Muennighoff et al. 2023; Luukkonen et al., *Poro 34B*, 2024.

Case study: Poro and Viking on LUMI

Poro 34B (2024) — Silo AI with TurkuNLP and HPLT, trained on LUMI on roughly **1T tokens**:

- English (SlimPajama), the majority of the mixture
- *all* the usable Finnish there was — on the order of 30B tokens, repeated several epochs
- a large code share (StarCoder)
- explicit **English–Finnish translation pairs**

Viking 7B / 13B / 33B — the follow-up: about 2T tokens across Finnish, English, Swedish, Danish, Norwegian, Icelandic and code.

What it bought

Finnish performance far beyond anything trainable on Finnish alone, and English did not suffer for it. That is the cross-lingual transfer argument, demonstrated at scale on a national supercomputer.

What I would do differently

The architecture was frozen roughly nine months before release, and the field moved. A token budget that looked ambitious at kick-off was unremarkable at launch. And a large share of the engineering effort went into the software stack rather than the model.

Source: Luukkonen et al., *Poro 34B and the Blessing of Multilinguality*, 2024; Viking model cards, Silo AI / TurkuNLP / HPLT, 2024. Mixture shares approximate.

Licensing, and the open-data question

What is actually released

- **weights only** — almost everything called “open”: Llama, Qwen, Gemma, DeepSeek, Mistral
- **weights + data + code + recipe** — OLMo, and very little else
- **data only** — FineWeb, Dolma, HPLT, The Stack

The legal status of training on copyrighted web text is unsettled and differs between the EU and the US. The EU’s text-and-data-mining exception has an opt-out that publishers increasingly exercise.

Why this matters for research

If the data is secret, the experiment is not reproducible, and claims about architecture cannot be separated from claims about data. That is the concrete reason I said on Tuesday that “which architecture is best” is unanswerable from public evidence.

For your project: name the licence of every model and dataset you use, and check that it permits academic use. This is in the rubric.

Take away

At the same scale, what separates two models is mostly what they were trained on, and that is the part of a recipe least likely to be documented.

Scaling laws

What scaling laws tell us

Across many experiments, three broad patterns appear:

1. a larger model can learn more complex patterns;
2. more training data gives the model more examples to learn from;
3. more compute usually lowers validation loss, but each additional improvement costs more than the previous one.

These trends are called **scaling laws**. They are fitted from smaller training runs and used to estimate what may happen at a larger scale.

Why this is useful

Before paying for a large run, a team can compare possible model sizes, data budgets and expected loss using smaller experiments.

Use them as forecasts, not guarantees about a model or task.

Source: Kaplan et al., *Scaling Laws for Neural Language Models*, 2020.

Model size and data size must be balanced

With a fixed compute budget, there is a trade-off:

- a very large model trained on too little text does not see enough examples;
- a very small model cannot make full use of an enormous dataset.

The Chinchilla result. A 70B-parameter model trained on 1.4T tokens outperformed a 280B-parameter model trained on 300B tokens, with roughly the same training compute.

The lesson. More parameters are not automatically better. Model size and training data should be chosen together.

The study's estimate of about 20 training tokens per parameter is a historical reference, not a universal rule.

Why this matters: scaling decisions determine both the training budget and the size of the model that must later be served.

Source: Hoffmann et al., *Training Compute-Optimal Large Language Models*, 2022.

A USEFUL COMPUTE ESTIMATE

A common approximation for dense transformer training is:

$$\boxed{C \approx 6ND} \qquad N = \text{parameters}, \quad D = \text{training tokens}$$

For a **20-million-parameter** Lab 1 model trained on **1 billion tokens**:

$$C = 6ND = 6 \times (2 \times 10^7) \times (1 \times 10^9) = 1.2 \times 10^{17}\ \mathrm{FLOPs}$$

What the estimate is used for

It lets us compare possible model and data sizes before training and gives a rough lower bound on the amount of computation required.

Real wall-clock time also depends on the GPU, software, data loading and how efficiently the hardware is used. Measure it in the lab rather than treating the formula as a runtime prediction.

Loss is useful, but it is not the final goal

Scaling laws predict validation loss. Product and research decisions usually depend on something else: task accuracy, reliability, latency, memory use or cost per request.

Two models with similar loss can differ because of their data mixture, tokenizer, context length and post-training. A small loss improvement may also be too expensive to justify at inference time.

How scaling laws are used

Use small runs to forecast whether a larger pretraining run is healthy and what loss it may reach. Then evaluate the resulting model on the tasks and constraints that matter for its intended use. Loss is a training diagnostic and forecasting target; it does not replace downstream evaluation.

Take away

$C \approx 6ND$ estimates what a run costs. It does not tell us whether the resulting model is useful.

Training at scale

Why training uses much more memory than inference

For inference, the main large object is the model itself. Training must keep several additional kinds of information:

- the model parameters;
- the gradients that show how the parameters should change;
- extra state used by the optimiser;
- intermediate activations needed for backpropagation.

As a rough example, a 7B-parameter model may need more than **100 GB** before all activation memory is counted. Exact requirements depend on the training setup.

Take away

A model that fits on one GPU for inference may still require several GPUs to train.

What changes when training spans many GPUs?

The work has to be divided across devices:

- different GPUs process different data or different parts of the model;
- they regularly exchange results over the network;
- adding GPUs reduces wall-clock time only if communication does not become the bottleneck.

Why this matters: scaling is a systems problem as well as a modelling problem.

Long runs also need recovery

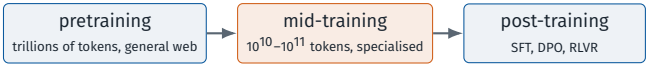
Large jobs run for days or months, so hardware and software failures are expected. The training system saves **checkpoints**: periodic copies of the model and training state from which a run can resume.

In the Llama 3 405B run, automated recovery kept effective training time above 90% despite hundreds of interruptions.

Source: Grattafiori et al., *The Llama 3 Herd of Models*, 2024; personal experience, Poro / Viking on LUMI, 2023–2024.

Mid-training

Mid-training: targeted continued pretraining



Same objective, different data. Mid-training is still next-token prediction. What changes is that the mixture is chosen deliberately rather than scraped.

Why it exists. The cheapest place to install a capability is usually the last few per cent of pretraining, rather than the start of a trillion-token run or a fine-tune on a hundred thousand examples.

The term only stabilised around 2024–2025. Before that the same work was called “annealing”, “the decay phase”, “continued pretraining” or nothing at all.

Source: OLMo 2 (Allen AI, 2024) is the clearest publicly documented example; Raschka, *The State of LLMs*, 2025, on the terminology.

Long-context extension: the problem

Pretraining at 128k context is wasteful: attention is quadratic, activation memory is linear in sequence length, and almost no documents are that long. So you pretrain short, at 4k or 8k, and extend afterwards.

Why you cannot just feed it more tokens. RoPE encodes position by rotating each attention dimension by an angle proportional to the token's position. Think of a bank of clocks, each ticking at a different speed, with the whole range set by a number called the **base**, usually 10^4.

Past the positions seen in training, the fast clocks have wrapped around many times into a combination of readings the model has never encountered. Attention degenerates.

Source: Su et al., RoFormer, 2021; Grattafiori et al. 2024.

Base scaling, the blunt fix

Increase the base: $10^4 \to 5 \times 10^5$ (Llama 3) or higher. Every frequency slows down, so the same position index maps to a smaller angle and more positions fit inside the range the model knows. It works, but you pay for it in resolution on *local* position, which is what the model leans on most.

A short final stage with higher-quality data

After broad pretraining, training can continue briefly on a smaller, carefully selected mixture.

The mixture may emphasise:

- high-quality explanatory text;
- mathematics or code;
- long documents;
- a target language or domain.

Why do this near the end?

It is cheaper than repeating the full pretraining run, and the same base checkpoint can be continued with different mixtures for different goals.

The trade-off

Focusing too strongly on one domain can reduce performance elsewhere, so the resulting model still needs broad evaluation.

Source: OLMo 2 (Allen AI, 2024).

What targeted data can add

What actually goes into a mid-training mixture, and why:

Ingredient	What it is for
maths	symbolic step chains; the best lever on reasoning
code	structure and verifiable chains; lifts reasoning
long documents	teaches the model to *use* its context window
instruction-like text	question-answer formatting for SFT to grip
target language	the cheapest place to add a new language
synthetic textbooks	dense explanatory prose; narrows the distribution

By the bottom two rows, mid-training is hard to tell apart from SFT without the loss masking.

Source: OLMo 2 (2024); Aryabumi et al., *To Code or Not To Code?*, 2024; Gunasekar et al., *Textbooks Are All You Need*, 2023.

What pretraining buys

In-context learning, live

Demo: a base model, with no instruction tuning and no chat template.

```
Review: The food was cold and the waiter ignored us.  Sentiment: negative
Review: Beautiful room, spotless, and the staff were kind.  Sentiment: positive
Review: It arrived late and the box was crushed.  Sentiment: negative
Review: Exactly what I wanted, and cheaper than the shop.  Sentiment:
```

The model completes `positive`. Nobody trained it to classify sentiment, and no weight was updated.

It picked up sentiment classification **from the prompt**, because the prompt is a document whose most likely continuation is the correct label.

In-context learning is the emergent capability that made the whole prompting era possible. You get it free from next-token prediction on a corpus that happens to contain lists, tables and worked examples.

Source: Brown et al., *Language Models are Few-Shot Learners*, 2020.

What in-context learning means

What happens. The prefix changes the model's *conditional distribution*. No weights move, and the effect disappears as soon as the context does.

What to be careful about. Calling it learning is a stretch. There is a real literature analysing it as gradient descent inside the forward pass, but the case is suggestive rather than settled.

Empirically: performance rises with the number of examples, with the consistency of the *format*, and surprisingly weakly with whether the labels in the examples are actually correct.

Why it emerges at all

The pretraining corpus is full of documents in which a pattern is established and then continued: glossaries, tables, translation pairs, exercise-and-answer sets, code with consistent conventions. A model that models those documents well *must* learn to infer a rule from a prefix and apply it. There was never a separate objective for this.

In Lab 1 Part D you run the same few-shot prompt on your tiny model and on a larger base model. Treat this as a qualitative comparison: scale, architecture, tokenizer and training data all differ.

Source: Brown et al. 2020; Min et al., *Rethinking the Role of Demonstrations*, 2022.

Now ask it a question

Demo: the same base model, same session, no few-shot examples.

```
Prompt:  What is the capital of Finland?

Output:  What is the capital of Sweden?
         What is the capital of Norway?
         What is the largest city in Finland?
         What is the population of Helsinki?
         Answer the following questions in complete sentences.
         1. What is the capital of Finland? _____________
         2. What is ...
```

Nothing has necessarily gone wrong here. More questions are one plausible continuation because question lists and worksheets occur in the pretraining corpus.

And it will not stop. No token in the corpus reliably means “the assistant has finished its turn”, because there is no assistant.

What the base model does not have

Missing	Why pretraining could not supply it
stopping	no end-of-turn token exists in web documents
turn-taking	no notion of “user” and “assistant” roles
instruction following	instructions occur, but obeying them is rarely the likeliest continuation
a preferred answer	good and bad answers appear at their web frequencies
refusal	declining a request is rare, and inconsistent where it occurs
a consistent persona	the corpus is millions of different authors

Every row is a problem of **selection** rather than **knowledge**. The few-shot demo showed the base model can produce the right behaviour; it just does not reach for it by default. That is the thesis of next Tuesday.

Source: Ouyang et al., *InstructGPT*, 2022, for the original framing.

What pretraining gives us

What pretraining produced

- a compressed model of a large slice of human text
- world knowledge, syntax, code semantics, arithmetic, discourse structure
- the ability to infer a task from a prefix and continue it
- a representation good enough that a few thousand examples can redirect its behaviour

What it did not produce

- reliable assistant-style turn-taking, instruction following or stopping behaviour

Why base models are still released

A base model is a useful starting point when researchers want to study or shape behaviour without an existing instruction-tuning policy. Lab 2 compares a base model with a corresponding instruct model. You may use a different suitable pair from the one used elsewhere in your project.

Take away

Pretraining supplies broad language and task capability. Post-training can select, strengthen and sometimes add behaviours using much less compute, but it still depends on what the base model can represent.

Lab 1 briefing

Lab 1: from tokenizer to tiny GPT

A. Tokenizer *CPU Jupyter, about 1 h*
Byte-level BPE on a Finnish Wikipedia slice at two or three vocabulary sizes, then compare fertility against Qwen3.5 and Gemma 4 on FLORES-200.

B. Tiny GPT *GPU batch, about 20 min / run*
Train a 10–30M-parameter decoder with *your* tokenizer. Script and Slurm template provided.

C. Scaling *3 GPU batch jobs*
Three model sizes at a fixed token budget. Plot validation loss against compute, estimating compute as $6ND$.

D. Tiny versus pretrained *GPU Jupyter, about 20 min*
The same few-shot prompt on your tiny model and on a real base model.

E. Project *planning*
Choose a task, model and held-out set. You may revise this later.

How the parts connect to the lectures

the training loop and the loss curve $\Rightarrow$ B

$C \approx 6ND$ and scaling laws $\Rightarrow$ C

in-context learning $\Rightarrow$ D

and Tuesday's tokenization $\Rightarrow$ A

A complete run of the three supplied model sizes should use roughly **one GPU-hour** in total.

Start with `lab1-example/README.md`; it includes a CPU smoke test, the training script, configurations and plotting code.

How to check that a run is working

Check	Expected pattern	If it differs
first-step loss	close to $\ln(V)$; about 10.4 for 32k tokens	check initialization, vocabulary and labels
training loss	falls quickly, then more slowly	inspect the learning rate and batches
validation loss	follows training loss	check the split and possible overfitting
throughput	becomes stable after warmup	inspect the data loader and GPU utilisation
output files	metrics, summary and weights exist	read the end of the job log

The one-line sanity check: your initial loss should be close to the log of your vocabulary size, and $\ln 32000 \approx 10.4$. If your run starts far from that value, investigate before spending GPU time.

Material for the final report

1. a **fertility table** covering your tokenizers and two production tokenizers, across three languages
2. **loss curves** for all your runs, on one set of axes
3. a **scaling plot** of validation loss against $6ND$, log–log
4. one paragraph comparing **your run's compute** with a published frontier run, using $6ND$ for both
5. your project plan: task, model and held-out set

What makes the analysis useful

A number with the method that produced it, and a sentence saying what you concluded from it. If a result surprises you, investigate it and record what you checked. "The three runs did not order the way I expected, and here is what I checked" is a good paragraph.

Keep the scripts, configurations and machine-readable result files in your repository. There is no separate Lab 1 submission.

Working on Lab 1

- Roihu is down for maintenance on 3–4 November. Those are not available for GPU work.
- **There is no supervised lab this week.** The next time we are all in a room with GPUs is 5 November.
- Questions go to the course channel. It is watched all week and answers are visible to everybody.
- There is no intermediate submission. Keep your work for the final report and code archive.

If something breaks

Post the **job id**, the **command**, and the **last twenty lines** of the log. Three quarters of problems are solved from that alone. Do not post screenshots of a terminal.

A useful starting point

Run the bundled CPU smoke test, then Part A. This checks your environment and data flow before you request GPU time.

Three things to remember

1. $C \approx 6ND$ **gives a useful first estimate of training compute.** Two FLOPs per parameter per token forward and about four backward. Remember what the approximation leaves out.
2. **Data choices strongly affect model quality.** Inspect the data for obvious problems, then compare pipeline choices by training small, controlled models and evaluating them.
3. **A deployed model may be trained past the compute-optimal point.** Extra training can produce a smaller model with lower inference cost, trading a one-time expense for repeated savings.

And one number to carry: **16 bytes per parameter** to train in mixed precision, against 2 to serve.

Tuesday: the model that will not stop talking

We ended on a base model that answered a question with more questions and then kept going.

Nobody patches that. It is the gap between a model of *documents* and an *assistant*, and closing it takes a whole discipline:

SFT → preference optimisation → RLVR → distillation

Each stage exists because the previous one has a specific limitation.

Tuesday 3 November

Post-training. Chat templates, loss masking, DPO, GRPO, and why being able to *check* an answer removed the need for a reward model.

The base-versus-instruct demo will be pre-recorded, because Roihu is down that day.

This week: begin Lab 1 and keep the outputs for your final report. There is no intermediate submission.

Reading

Core

- Stanford CS336, *Language Modeling from Scratch*, the lectures on data and on systems
- Hugging Face, *The Smol Training Playbook*, 2025
- Karpathy, *nanoGPT*

Data

- Penedo et al., *The FineWeb Datasets*, 2024
- Lozhkov et al., *FineWeb-Edu*, 2024
- Soldaini et al., *Dolma*, 2024
- Luukkonen et al., *Poro 34B and the Blessing of Multilinguality*, 2024

Scaling

- Kaplan et al. 2020; Hoffmann et al. 2022 (Chinchilla)
- Muennighoff et al., *Scaling Data-Constrained Language Models*, 2023
- Schaeffer et al., *Are Emergent Abilities a Mirage?*, 2023

Systems

- Rajbhandari et al., *ZeRO*, 2020
- Shoeybi et al., *Megatron-LM*, 2019
- Grattafiori et al., *The Llama 3 Herd of Models*, 2024; read section 3 on infrastructure
- DeepSeek-AI, *DeepSeek-V3 Technical Report*, 2024, for fp8 training in detail

For current developments, Raschka's *Ahead of AI*: *A Dream of Spring for Open-Weight LLMs* (Feb 2026) and *The State of LLMs* (2025).